

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
School of Computer Science and Engineering

PROJECT ASSIGNMENT
Topics, Requirements and Assessment

Course: IT079IU – Principles of Database Management
Semester 1, Academic Year 2026–2027

Instructor: Assoc. Prof. Dr. Nguyen Thi Thuy Loan
nttloan@hcmiu.edu.vn · Office O1.610

Part 1. What the project is

The project is where the whole course comes together. Over the semester, your group takes one application domain from requirements in plain language all the way to a working relational database: an ER diagram, a relational schema derived from it, a set of functional dependencies, a normalized design, the SQL that creates it, sample data, and the queries that answer real questions about that data.

The project has two halves that are marked together. The first is the database itself: you design it, create it, fill it with data, query it, and update it. The second is a Java desktop application that connects to that database and puts a form in front of it, so that a user who knows no SQL can search, read and change the data through the screen.

The application is a window onto the database, not a separate piece of software. It is not marked by how it looks. It is marked on whether every screen is driven by real SQL running against your own schema, whether the constraints you declared are respected when the user does something wrong, and whether the group can explain the statement behind each button.

1.1 What the project assesses

Each stage of the project corresponds to a lecture, so the work follows the course week by week:

Project stage	Lecture
Requirements and business rules	Lecture 1
ER diagram with constraints	Lecture 2
Mapping the diagram to a relational schema	Lecture 3
Queries expressed in relational algebra	Lecture 4
CREATE TABLE, constraints and sample data	Lecture 5
Queries in SQL: joins, grouping, nesting	Lecture 6
Functional dependencies and keys	Lecture 7
Normalization to 3NF or BCNF	Lecture 8

1.2 Groups and topics

Forming a group

- Form your own group of ten or eleven students and register it before the end of week 2. The exact size depends on the size of the class and is announced with the registration link.
- Anyone not registered in a group by that deadline is placed in one by the instructor, drawn at random from the class list. A group formed this way carries exactly the same obligations as one formed by choice.
- Once registered, the membership of a group is fixed. A change of group afterward requires the written agreement of both groups and of the instructor.

Choosing a topic

- Each group chooses its own topic from the twenty-five in Part 3. Read several topics before deciding: they differ in how much data must be modeled and in how demanding the last query is.
- No two groups in the same class may work on the same topic. Topics are taken on a first-come, first-served basis, and the registration sheet shows in real time which ones are still free.
- A group may instead propose a topic of its own. The proposal must be submitted in writing, must satisfy every minimum requirement of section 2.2, must not duplicate the domain of another group, and takes effect only once the instructor has approved it.
- Confirm your topic - chosen or proposed - before the end of week 3, which is also when the project proposal (D1) is due. A group that has not confirmed one by then is given a topic by the instructor from those still available.
- After approval, a topic may be changed only with the instructor's consent, and no later than the progress report in week 5.

Registration

- Register your group and topic in the Google Sheet provided through the Zalo group. For each member, enter their name, student ID, and phone number, together with the selected topic.
- Include a copy of the group information (member names, student IDs, phone numbers, and topic) with every project deliverable.

1.3 Working as a group

The project is group work, but the grade is individual: Members of the same group may receive different marks based on their individual contributions. Divide the work by role and record who did what, week by week, in the work-allocation Google Sheet.

For example, the following division of roles has worked well in previous cohorts. It is a suggestion, not a rule. For a group of ten or eleven, some roles may be shared by two members, depending on the workload.

Role	Suggested members	Responsibility
Group leader	1	Schedule, deadlines, submissions, contact with the instructor
Data modeller	2	Business rules, ER diagram, mapping to the schema
Normalization lead	1–2	Functional dependencies, keys, normal forms, decomposition
Database developer	1–2	CREATE TABLE script, constraints, sample data, indexes
Query developer	1–2	SQL queries and relational-algebra versions
Java developer	2	JDBC layer, forms, validation, error handling
Tester and writer	1	Testing, constraint checking, integration of the final report

Two rules apply, whatever the division of labour. Every member must take part in the ER modeling discussion in the first three weeks, because that is where the course content is. And every member must be able to explain the whole design, not only their own part; at the presentation each member is questioned individually, and an answer of “that was someone else’s section” costs marks.

Part 2. Requirements

2.1 Deliverables and deadlines

Deliverable	Due	Length
D1. Project proposal	Week 3	1–3 pages
D2. Progress report: ER diagram and draft schema	Week 5	3–5 pages
D3. Final report	Week 10	25+ pages
D4. Presentation and demonstration	Weeks 13–15	15 minutes
D5. SQL script and sample data	With D3	.sql file
D6. Java application, source code and build instructions	With D3	project folder

Deadlines are fixed. Late submission is accepted only for a documented reason such as illness, agreed with the instructor before the due date. Another examination or another assignment is not a valid reason.

2.2 Minimum technical requirements

The final project must satisfy all of the following minimum technical requirements. A design that does not meet these requirements must be revised before the final submission can be assessed.

- At least eight entity sets and at least eight relationships in the ER diagram.
- At least one weak entity set, with its identifying relationship and its partial key.
- At least one many-to-many relationship carrying its own attribute.
- At least one of: an ISA hierarchy with its disjointness and completeness constraints, or a multivalued attribute, or a ternary relationship.
- At least one relationship of an entity set with itself, with role names on both edges.
- Cardinality and participation marked on every relationship.
- A complete set of functional dependencies, a minimal cover, and every candidate key derived and justified.

- Every relation in 3NF at least. If a relation is left in 3NF rather than BCNF, say which dependency would become unenforceable in BCNF and why you chose to stop.
- Every table created by an SQL script, with primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and ON DELETE and ON UPDATE action on every foreign key.
- At least fifteen rows of realistic sample data in every table, and enough data for every query below to return a non-empty result.
- At least twelve SQL queries: at least one multi-table join, one outer join, one aggregation with GROUP BY and HAVING, one nested query with IN or EXISTS, one query using NOT EXISTS to express "for all", one set operation, and one view.
- Four of those queries also written in relational algebra.

2.3 Building the database in SQL

Once the design in Section 2.2 is settled, implement it as a working relational database before developing the Java application. The database design and implementation remain the primary focus of this project.

Enforcing business rules

Business rules should be enforced at the database level whenever practical, using appropriate mechanisms such as PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and other DBMS-supported constraints. Rules that cannot reasonably be enforced with declarative constraints may be implemented with triggers, transactions, stored procedures, or application-level validation, provided the chosen approach is clearly documented and justified.

Students are assessed on whether each business rule is correctly enforced and justified, rather than on the specific implementation mechanism used.

Choice of database system

- Any SQL-based relational system may be used: MySQL or Microsoft SQL Server. MySQL 8.0 or later is the recommended default and is the system used in the laboratory sessions, but it is a recommendation, not a requirement.

- Whichever system you choose, it must actually enforce what you declare: primary keys, foreign keys with referential actions, UNIQUE and CHECK constraints, and transactions. Check this before you commit to a system - in MySQL.
- Name the system and its version in the report and in the README, and write your SQL in standard form wherever possible, so that the marker can read it without knowing your dialect. Where you rely on something specific to your system, say so in a comment.
- Any suitable graphical tool may be used to create or edit the database diagram, including reverse-engineering it from the implemented schema

What to submit

- Submit one script that runs on an empty server from top to bottom: drop the database if it already exists, create it, then the CREATE TABLE statements in an order that satisfies the foreign keys, then the INSERT statements. It must run twice in a row without manual repair.
- Declare every constraint in that script: primary keys, foreign keys with ON DELETE and ON UPDATE, NOT NULL, UNIQUE, CHECK and DEFAULT. A rule enforced only by your Java code does not count as a constraint and earns no marks.
- Submit the twelve queries of section 2.2 as a second, commented script, each preceded by the question it answers in English.
- Include the update statements the application needs: at least one INSERT, one UPDATE and one DELETE for each main table, and at least one operation that changes two tables together and therefore runs inside a transaction with COMMIT and ROLLBACK.
- Create an index on at least one column your queries search on, and say in the report which query it is meant to help.

Test the constraints before you build the interface: try to insert a duplicate key, a foreign key that does not exist, and a value that breaks a CHECK. If the system accepts any of them, the schema is not finished.

2.4 The Java application

The application is written in Java and connects to the database of section 2.3 through JDBC, using the driver for the system you chose - MySQL Connector/J, the PostgreSQL JDBC driver, and so on. Swing or JavaFX may be used for the forms; the choice is yours and carries no marks either way. Every screen must read from, or write to, the database through SQL that you wrote - no data may be held in a hard-coded array or a text file.

The application must contain at least

- A connection layer that opens the database from settings held outside the code, so the marker can point it at their own server without recompiling.
- A browse form that loads rows from a table into a grid, with the number of rows shown.
- A search form where the user types or selects criteria and the rows are filtered by a parameterised query.
- A master–detail form: choosing a row in the upper grid loads the related rows in the lower one, for example an order and its lines.
- An editing form that inserts, updates and deletes rows, with confirmation before a delete.
- A report screen that shows the result of an aggregate query, for example a total per month or per category.
- At least four of the five queries listed under your topic in Part 3, reachable from the interface.

Rules the code must follow

- Every statement that takes a user value must be a PreparedStatement with parameters. String concatenation into SQL is treated as a defect, and the marker will test it with an apostrophe in a text field.
- Constraint violations must be caught and explained to the user in plain language. Deleting a customer who still has orders must produce a readable message, not a stack trace.
- Input must be validated before it reaches the database: required fields, numeric ranges, date order.

- Connections, statements and result sets must be closed. Use try-with-resources.
- The insert, update and delete screens must be demonstrated live: the marker will change data and then ask you to show the change in the database itself.

Submit the source code as a folder, together with a README that gives the JDK version, the JDBC driver used, the connection settings and the steps to build and run. A project the marker cannot run is marked as if the screens do not exist.

2.5 Structure of the final report

The final report must contain the following sections, in this order:

1. Introduction: the domain, who uses the database, and what decisions it supports.
2. Requirements and business rules, written as numbered statements in plain language.
3. ER diagram, with a paragraph explaining every design decision that was not obvious.
4. Relational schema derived from the diagram, showing which mapping rule produced each table.
5. Functional dependencies, minimal cover and candidate keys.
6. Normalization: the normal form of each relation, and every decomposition with proof that it is lossless.
7. Database implementation: the database system and version used, the SQL script, the sample data, the transaction and the index, with the reason for each.
8. Queries: each query in English, in SQL, and its result on the sample data.
9. The application: how it connects to the database, a screenshot of every form, and the SQL behind each screen.
10. Discussion: what the design cannot express, what you would change, and the limits of the current schema.
11. Work allocation: who did what, agreed and signed by every member.
12. References.

2.6 Academic honesty and use of AI tools

You may use AI assistants to learn, to check syntax and to review your writing. You may not submit a design you cannot explain. Any section produced with substantial AI

assistance must be declared in an appendix, stating what was generated and how you verified it. Every member of the group must be able to explain and defend the whole design during the presentation; the presentation is where individual understanding is checked.

Copying a design from another group, from a previous cohort or from a website, without attribution, is plagiarism and is dealt with under the university regulations.

2.7 Assessment

The project component is worth 20% of the overall course grade and consists of two parts:

1. Own Project – 70%

Up to 70% of the project mark is awarded for the group's own project, including the project work, written report, presentation and demonstration, and responses to questions about the project. The Own Project is assessed using the rubric below.

Criterion	Marks
Motivation: clarity and significance of the problem	10
Requirements and related work: clear business rules and relevant sources	15
Technical work – see the breakdown below	45
Presentation, demonstration, and responses to questions	20
Quality of writing	10
Total	100

The score from this rubric is converted to 70% of the overall project mark.

2. Discussion – 30%

The remaining 30% is assessed individually based on each student's participation in the discussions during the three final project-presentation sessions. Students are expected to engage with other groups' projects by asking relevant questions, offering meaningful comments, and contributing constructively to the discussion.

Students who participate more actively and demonstrate better understanding and critical thinking may receive higher marks for this component.

2.8 Individual contribution

Each member's contribution to the group project is assessed individually. The instructor considers the work-allocation record, each member's contribution to the presentation and demonstration, and their ability to explain and answer questions about the work they were responsible for.

Members of the same group may therefore receive different marks for the Own Project component if their contributions or demonstrated understanding differ.

Part 3. Topics

Each topic below is described in five parts. Context says what the system is for. Objective states what the finished system must achieve; it is the sentence your proposal in D1 has to argue for, and the one the marker returns to at the demonstration. Data to model lists the information the database must hold; it is a starting point, not a complete specification, and you are expected to add what the domain needs. Modeling points name the design decisions that are known to be difficult in that domain and that the marker will look at first. Queries the database must answer are the questions your final schema must be able to answer; they are deliberately ordered from simple to demanding, and the last one usually needs a nested query or a division.

Reading the topic is not enough. Before you draw anything, write down the business rules of your domain as numbered sentences, and check each of them against your diagram afterward.

1. Railway Network and Timetables

Model the passenger railway network of a country: the stations, the tracks that join them, the trains, and the timetable that says which train calls at which station and when. Assume every train completes its journey within one day.

Objective. Build a system that manages railway operations by tracking stations, trains and schedules accurately, and that can be extended to seat reservations and ticketing.

Data to model

- Stations with a code, a name and a city.
- Track sections, each joining two stations, with a length in kilometres.
- Trains with a number, a name and a type (express, local, night).
- Timetable entries: for each train and each station it calls at, the stop order, the arrival time and the departure time.
- Coaches of each train, their class (sleeper, air-conditioned, general) and the seats in each coach.
- Bookings: a passenger, a train, a travel date, a seat and a fare.

Modelling points to get right

- A timetable entry is identified only inside its train — the stop number 3 means nothing on its own.
- One train calls at many stations and one station is served by many trains, and the stop order is a fact about the pair.
- A seat belongs to a coach and a coach belongs to a train.

Queries the database must answer

- List every station where train SE7 stops, in the order it calls at them.
- How many trains depart from Saigon station between 06:00 and 12:00?
- For each train, the total distance of its route.
- Which trains call at every station on the Saigon–Nha Trang line?
- Find the passengers holding two bookings on the same date.

2. Payroll Management

Model the monthly payroll of an organization: who is employed, what they are paid, what is added and what is deducted, and what the organization actually paid in each month.

Objective. Produce a payroll system that computes net pay correctly from basic pay, allowances and deductions, and keeps an auditable history for every employee and department.

Data to model

- Employees with an identifier, name, date of joining, job title and the department they belong to.
- Departments, each managed by exactly one employee, with the date the manager took over.
- Salary grades, each with a basic pay and the allowances attached to it.
- Allowance and deduction types (housing, transport, income tax, social insurance, unpaid leave).
- Payslips: one per employee per month, with the basic pay, each allowance line, each deduction line, and the net amount.
- Payment records: date paid, bank account, status.

Modeling points to get right

- A payslip line is identified only inside its payslip.
- An employee has many allowance types, and an allowance type applies to many employees, with an amount that depends on the pair.
- The employee who manages a department is also an employee - a relationship of a set with itself.

Queries the database must answer

- The payslip of one employee for one month, with every line.
- The total payroll cost of each department last month.
- Employees whose net pay fell compared with the previous month.
- The average net pay per job title, for titles with at least three employees.
- Employees who have received every allowance type available in their grade.

3. Service Marketplace

Model a platform where independent merchants publish services and customers book and pay for them. The platform itself takes a commission and must be able to report on both sides of every transaction.

Objective. Create a marketplace that gives merchants a digital storefront and lets customers book and pay for services, while the platform can report on transactions and commission from both sides.

Data to model

- Merchants with a profile, a category and a verification status.
- Services offered by a merchant: title, description, price, duration and whether it is currently offered.
- Customers with an account, one or more addresses and one or more phone numbers.
- Bookings: a customer books a service for a date and time, with a status (requested, confirmed, completed, cancelled).
- Payments and commission: amount, method, date, and the share retained by the platform.
- Reviews left by a customer for a completed booking: a score and a comment.

Modelling points to get right

- A customer has several phone numbers.
- A review exists only for a booking that has been completed.
- The booking carries the price actually charged, which may differ from today's list price.

Queries the database must answer

- The services of one merchant with their average review score.
- Total commission earned by the platform per month.
- The customers who have booked services from more than three different merchants.
- Merchants with no booking at all in the last 60 days.
- The category in which the average booking value is highest.

4. E-Commerce for Automotive Parts

Model an online shop selling car and motorbike parts. A part fits only certain vehicle models, and that compatibility is the core of the catalogue.

Objective. Deliver a catalogue in which a customer can find exactly the parts that fit their vehicle, and the seller can follow stock, orders and revenue.

Data to model

- Parts with a code, name, brand, unit price and current stock level.
- Categories arranged in levels (engine → filters → oil filters).
- Vehicle models with make, model name and year range.
- Compatibility: which part fits which vehicle model, with a note.
- Customers, their addresses and their orders.
- Order lines: the part, the quantity and the price at the time of the order.
- Shipments and payments linked to an order.

Modelling points to get right

- An order line is identified only inside its order.
- A part fits many vehicle models, and a vehicle model accepts many parts.
- A category may contain sub-categories — a relationship of a set with itself.

Queries the database must answer

- Every part that fits a Toyota Vios 2019.
- The ten best-selling parts by revenue last quarter.
- Orders that contain a part now out of stock.
- Customers who bought parts from every brand the shop carries.
- For each category, the number of parts and the average price.

5. At-Home Vehicle Servicing

Model a workshop that sends mechanics to the customer's address. Bookings must be matched to a mechanic who has the right skill and a free time slot.

Objective. Simplify doorstep servicing by matching each booking to a mechanic with the right skill and a free slot, and by billing labour and parts accurately.

Data to model

- Services with a name, a description, a standard duration and a price.

- Mechanics with their skills and the districts they cover.
- Working slots: for each mechanic and each date, the hours available.
- Bookings: a customer, an address, a date and slot, one or more services, and an assigned mechanic.
- Parts consumed during a job, with quantity and price.
- Invoices and payments; job completion reports with notes and photographs.

Modeling points to get right

- A working slot is identified only by a mechanic and a date.
- A booking may include several services, and each service in a booking has its own price and duration.
- A mechanic has several skills, and a skill is held by several mechanics.

Queries the database must answer

- The bookings of one mechanic for tomorrow, in time order.
- The mechanics qualified for an oil change who are free on Saturday morning.
- Revenue per service type in the last three months.
- Customers who have booked the same service more than twice.
- Jobs where the parts used cost more than the labour charged.

6. Project and Task Management

Model a tool where teams plan projects, break them into tasks, assign them and follow their progress. Tasks may depend on other tasks and may be discussed in comments.

Objective. Support teamwork with a database that tracks projects, tasks, assignments and dependencies, and that can always answer who is blocked and why.

Data to model

- Users, teams, and the membership of users in teams, with a role.
- Projects with a name, an owner, a start date, and a due date.
- Tasks within a project: title, description, priority, status, estimate, and due date.
- Assignment of tasks to users, with the date assigned.

- Dependencies: a task may not start before certain other tasks are finished.
- Comments on tasks, and attachments on comments.
- Labels applied to tasks.

Modeling points to get right

- A task depends on other tasks - a relationship of a set with itself.
- A user belongs to several teams and a team has several users, with a role that depends on the pair.
- An attachment exists only inside its comment.

Queries the database must answer

- Every open task of one project, ordered by due date.
- For each user, the number of tasks they have open and the number overdue.
- Tasks that cannot start because one of the tasks they depend on is unfinished.
- Projects where every task has been completed.
- The user who has commented on tasks in the largest number of different projects.

7. Airport Operations

Model the daily operation of an airport: scheduled flights, the passengers on them, the crew assigned, and the baggage carried.

Objective. Manage passengers, flights, crew and baggage in one schema so that a flight on a given date can be reconstructed completely, including every bag and every crew duty.

Data to model

- Airports, airlines and aircraft with their type and seat layout.
- Scheduled flights: number, route, departure and arrival times, aircraft used.
- Flight instances: the same flight number operates on many dates, each with a real departure time and a status.
- Passengers, their travel documents and their bookings.
- Seat assignments and boarding passes.

- Baggage items linked to a passenger and a flight instance, with weight and a status (checked in, loaded, delivered, lost).
- Staff (pilots, cabin crew, ground staff) and the duties they are assigned to.

Modeling points to get right

- A flight instance is identified by the flight number plus the date.
- A seat is identified only inside its aircraft.
- Crew members are assigned to many flight instances, and each flight instance has many crew members, with a role.

Queries the database must answer

- The passenger list of flight VN220 on a given date.
- The flights delayed by more than 30 minutes last week.
- Total baggage weight carried by each aircraft yesterday.
- Crew members who worked on every flight on a given route in a month.
- Passengers whose baggage was recorded as loaded but never as delivered.

8. Insurance Policies and Claims

Model an insurer that sells several kinds of policies to individuals and to companies, collects premiums, and settles claims.

Objective. Keep policies, premiums, and claims consistent for individual and corporate clients, and make the profitability of each product visible.

Data to model

- Clients, who are either individuals or companies, with the attributes specific to each kind.
- Products: life, health, vehicle and property insurance, each with its own conditions.
- Policies: a client buys a product for a period, through a branch, with a premium and a sum insured.
- Beneficiaries named on a policy.
- Premium invoices and the payments made against them.

- Claims: incident date, reported date, amount claimed, amount settled and status.
- Assessors assigned to claims.

Modeling points to get right

- Individual and company clients form a hierarchy under a common client identifier.
- A claim is identified only inside its policy.
- A policy may name several beneficiaries with a percentage share.

Queries the database must answer

- All active policies of one client with their next payment date.
- The total amount claimed against each product this year.
- Policies whose claims already exceed the premiums collected.
- Clients who hold at least one policy of every product the company sells.
- Claims still unsettled sixty days after being reported.

9. Vehicle Registration Authority

Model the register of vehicles kept by a transport authority: who owns what, since when, whether the vehicle is insured and inspected, and what fines are outstanding.

Objective. Track vehicle ownership over time and link it to registration, insurance, inspection and fines, so that authorities can check compliance from one record.

Data to model

- Vehicles with a plate number, chassis and engine number, make, model, year and colour.
- Owners, who may be individuals or organisations.
- Ownership periods: one vehicle passes from owner to owner over time.
- Registrations and their renewal dates.
- Insurance policies covering a vehicle for a period.
- Technical inspections, with the date, the result and the inspector.
- Traffic violations and the fines issued, with a payment status.

Modelling points to get right

- The same vehicle has several owners over time, so the ownership period is an entity of its own.
- A fine is attached to a violation, and a violation is attached to a vehicle and a date.
- Individuals and organisations form a hierarchy under a common owner identifier.

Queries the database must answer

- The full ownership history of one plate number.
- Vehicles whose registration expires within thirty days.
- Owners with more than one unpaid fine.
- Vehicles that have passed every inspection they have ever had.
- For each district, the number of vehicles registered and the share currently insured.

10. Blood Donation and Blood Banks

Model a network of blood banks: who may donate, what is currently in stock, which hospital has requested what, and how requests are met.

Objective. Coordinate donors, blood banks, and hospital requests so that availability by group and component is transparent and no units are issued after expiry.

Data to model

- Donors with a blood group, contact details and a donation history.
- Blood banks with locations, opening hours, and staff.
- Donation events: the donor, the bank, the date, the volume taken and the screening result.
- Blood units in stock, each with a group, a component (whole blood, plasma, platelets), a collection date and an expiry date.
- Hospitals and the requests they issue: group, component, quantity, urgency and date.
- Issue records showing which units were released against which request.

Modeling points to get right

- A blood unit is identified only inside the bank that holds it.
- A request may be satisfied by several units and a unit is issued only once.

- A donor may not donate again before a minimum interval has passed.

Queries the database must answer

- Units of group O – that expire within seven days.
- The donors eligible to donate again today.
- Requests that were only partly filled last month.
- Blood banks that hold at least one unit of every group.
- The average number of days between collection and issue, per component.

11. Corporate Training Programs

A company manages training courses for its employees. The database records instructors, courses, training sessions, and trainees. Each course may be offered in multiple sessions; instructors may teach different sessions; and trainees may enroll in appropriate sessions. The system should support the management of course offerings, instructor assignments, trainee enrolments, and training records.

Objective. Manage courses, instructors, and trainees while respecting the constraints on teaching-team size, instructor workload, and session capacity.

Data to model

- Instructors with their areas of expertise.
- Courses with a code, a title, a level and prerequisites.
- Teaching teams: each course is delivered by a team of at least five instructors, and no instructor belongs to more than three teams.
- Sessions: a course runs on specified dates at a specified location, with a capacity.
- Trainees and their enrolments: a trainee takes exactly one course per session.
- Attendance, assessments and certificates issued.

Modeling points to get right

- A course may require another course as a prerequisite - a relationship of a set with itself.
- The constraints on team size and on the number of teams per instructor cannot be drawn in the diagram; state them separately.

- A session is identified by its course plus a sequence number.

Queries the database must answer

- The instructors teaching a given course this quarter.
- Sessions that are more than 90% full.
- Trainees who have completed every prerequisite of the course they are enrolled in.
- The instructor teaching in the largest number of teams.
- Average assessment score per course, for courses run at least twice.

12. Personal Income Tax Administration

Model the tax file of individual taxpayers: the income they declare, the deductions they claim, the tax computed under the current bands, and what has been paid.

Objective. Compute tax accurately from declared income and allowed deductions under the bands of each year, and keep filings and payments traceable.

Data to model

- Taxpayers with a tax code, a residency status, and dependants.
- Employers and the income they report for each taxpayer.
- Income items by type: salary, business, investment, rental, other.
- Deductions: personal allowance, dependants, insurance contributions, charitable donations.
- Tax bands, which change from year to year.
- Tax returns filed for a year, the amount assessed, and the payments made against them.

Modelling points to get right

- A dependant is identified only inside the taxpayer who claims them.
- A tax band belongs to a tax year, so the same band number means different rates in different years.
- A taxpayer may have income from several employers in one year.

Queries the database must answer

- The total declared income of one taxpayer for a year, by type.
- The tax payable for that taxpayer under the bands of that year.
- Taxpayers who filed late in each of the last three years.
- The number of taxpayers in each band, per year.
- Taxpayers who claim a dependant also claimed by another taxpayer.

13. Water Supply Billing

Model a water utility: the connections it serves, the meter readings it collects, the bills it issues under a tiered tariff, and the complaints it receives.

Objective. Turn meter readings into correct bills under a tiered tariff, and give the utility a clear view of consumption, arrears and unresolved complaints.

Data to model

- Customers: households, businesses and institutions.
- Connections, each at an address, with an installation date and a status.
- Meters installed on a connection, and the periods during which each meter was in place.
- Meter readings: date, reading, and who took it.
- Tariff tiers: price per cubic metre, by consumption band and customer type, valid from a date.
- Bills computed for a period, and the payments made against them.
- Complaints and the maintenance work orders they generate.

Modelling points to get right

- A meter reading is identified only inside its meter.
- A tariff has a validity period, so the price applied depends on the billing period.
- A connection may have had several meters over its life.

Queries the database must answer

- The consumption of one connection over the last twelve months.
- The bill for one connection for one period, showing each tariff tier applied.

- Connections whose consumption doubled compared with the same period last year.
- Unpaid bills older than sixty days, grouped by district.
- Complaints that had not produced a work order within a week.

14. University Health Service

Model the health service of a university: student records, consultations, prescriptions, vaccinations and the monitoring of contagious disease.

Objective. Give the health service reliable access to student medical records and let it detect and follow up outbreaks early.

Data to model

- Students and staff registered with the service, with an allergy list.
- Medical staff with their speciality and consulting hours.
- Appointments: who, with whom, when, and the outcome.
- Consultations: symptoms recorded, diagnosis and notes.
- Prescriptions: the medicines, the dose and the duration.
- Vaccination records.
- Notifiable cases: disease, date reported, and the contacts traced.

Modelling points to get right

- A prescription line is identified only inside its prescription.
- A patient has several allergies and an allergy affects several patients.
- Students and staff form a hierarchy under a common patient identifier.

Queries the database must answer

- The consultation history of one patient.
- The doctors with free appointment slots this week.
- The five most frequent diagnoses this semester.
- Patients who received every dose in a vaccination course.
- Cases of one disease reported in the same building within fourteen days.

15. Employee Attendance and Shift Scheduling

Model attendance in an organization that works in shifts: who is rostered when, who actually attended, and how leave is requested and approved.

Objective. Give managers accurate attendance and overtime data from rosters, clock events and approved leave, so staffing decisions rest on facts.

Data to model

- Employees, departments and job titles.
- Shift patterns: a shift has a name, a start time and an end time.
- Rosters: which employee is assigned to which shift on which date.
- Clock events: time in and time out, with the device or location.
- Leave types and leave balances per employee per year.
- Leave requests with an approval workflow and an approver.
- Public holidays.

Modelling points to get right

- A roster entry is identified by the employee plus the date plus the shift.
- The approver is also an employee - a relationship of a set with itself.
- A leave balance belongs to an employee and a year.

Queries the database must answer

- The roster of one department for next week.
- Employees who clocked in late more than three times last month.
- Total overtime hours per department this quarter.
- Leave requests still awaiting approval after five working days.
- Employees who were present on every working day of the month.

16. Online Voting

Model an electronic election: who is entitled to vote, in which contests, and how the system guarantees that each entitled voter votes at most once while the ballot itself stays anonymous.

Objective. Guarantee that every eligible voter votes at most once while the ballot stays anonymous, and that results and turnout can be verified afterward.

Data to model

- Voters with an identifier, a constituency, and an eligibility status.
- Elections, and the contests within an election (presidential, council, referendum).
- Candidates standing in a contest, with a party.
- Ballots cast: a ballot belongs to a contest and records a choice.
- Voting records: which voter has voted in which contest, kept separately from the ballots.
- Polling periods, and audit entries for every administrative action.

Modelling points to get right

- The separation between the voting record and the ballot is the central design decision - explain how it prevents double voting without linking a voter to a choice.
- A candidate stands in exactly one contest; a contest has many candidates.
- A voter is eligible for the contests of their constituency only.

Queries the database must answer

- The result of one contest, by candidate.
- Turnout per constituency.
- Voters entitled to vote in a contest who have not yet voted.
- Contests in which every candidate received at least one vote.
- Any voter recorded as having voted twice in the same contest - the query that must always return nothing.

17. Video-on-Demand Platform

Model the catalogue and the viewing history of a streaming service. The video files themselves are not stored in the database; the metadata, the subscriptions and the viewing behaviour are.

Objective. Model a catalogue and viewing history rich enough to answer what subscribers watch, how far they watch it, and what the service should recommend next.

Data to model

- Titles: films and series, with a year, a synopsis and a rating.
- Series structure: seasons and episodes, each episode with a number and a duration.
- People: actors and directors, and the roles they played in a title.
- Genres and languages applied to titles.
- Subscribers, their plan, and the profiles inside one account.
- Viewing events: which profile watched which episode, when, and how far.
- Watch lists and ratings given by profiles.

Modeling points to get right

- An episode is identified by its series, its season number and its episode number.
- A person may act in a title and also direct it, and the role belongs to the pair.
- A profile is identified only inside its account.

Queries the database must answer

- Every episode of one series in order, with its duration.
- The ten most watched titles last month.
- Subscribers who have not watched anything for ninety days.
- Profiles that have watched every episode of a given season.
- The average completion rate per genre.

18. Research Article Repository

Model a repository of research articles with their authors, venues, keywords, and citations - the structure a literature search tool needs.

Objective. Organize articles, authors, venues, and citations so that a researcher can trace the literature in both directions from any article.

Data to model

- Articles with a title, an abstract, a year and a DOI.

- Authors with an affiliation that may change over time.
- Authorship, including the position of each author in the list.
- Venues: journals and conferences, with their issues or editions.
- Keywords and subject areas assigned to an article.
- Citations: an article cites other articles.
- Users, their saved searches and their reading lists.

Modeling points to get right

- Citation is a relationship of the article set with itself.
- The author position is a fact about the pair (article, author), not about either alone.
- An affiliation holds for an author during a period.

Queries the database must answer

- The reference list of one article.
- The articles that cite a given article.
- The ten most cited articles in one subject area.
- Authors who have published in every venue of a given list.
- For each year, the average number of authors per article.

19. Retail Invoicing

Model the billing counter of a retail chain: products and their prices over time, sales invoices, discounts, returns and the daily takings of each shop.

Objective. Bill customers quickly and correctly, keeping historical prices on past invoices, and let the chain compare takings across shops and cashiers.

Data to model

- Shops, counters and cashiers.
- Products with a barcode, a unit and a tax rate.
- Price history: a product has a price valid from a date, so old invoices keep the old price.
- Invoices: shop, cashier, date and time, customer if known.

- Invoice lines: product, quantity, unit price, discount.
- Promotions and the rules that trigger them.
- Returns linked to the original invoice line.
- Payments by cash, card or e-wallet, possibly split across methods.

Modeling points to get right

- An invoice line is identified only inside its invoice.
- One invoice may be paid by more than one method.
- The price on the invoice line must be the price valid on the invoice date, not today's price.

Queries the database must answer

- A full invoice with its lines, discounts and total.
- Daily takings per shop for the last week.
- Products whose price rose more than 10% this year.
- The cashier with the highest average basket value.
- Invoices where the returned amount exceeds half the original total.

20. Online Examination System

Model an examination platform: the question bank, the papers assembled from it, the attempts made by students, and the marks awarded.

Objective. Assemble papers from a question bank, record attempts and marks reliably, and show which topics students understood least.

Data to model

- Courses and the topics within each course.
- Questions with a type (multiple choice, short answer), a difficulty and a topic.
- Options for a multiple-choice question, with the correct one marked.
- Examinations: a course, a date, a duration and the rules for attempts.
- Papers: which questions appear on an examination, in which order, for how many marks.

- Students, their registrations and their attempts, with the start and end time.
- Answers given, and the marks awarded per answer.

Modeling points to get right

- An option is identified only inside its question.
- The mark of a question depends on the examination in which it appears.
- A student may be allowed several attempts; each attempt is an entity of its own.

Queries the database must answer

- The paper of one examination, with the marks of each question.
- The score of one student in one attempt.
- Questions answered incorrectly by more than 70% of students.
- Students who attempted every examination of a course.
- The average score per topic, to show which topic was least understood.

21. Warehouse Management

Model a warehouse: what is stored where, what arrives, what leaves, and how stock levels are kept correct across several locations.

Objective. Keep stock levels correct across bins and warehouses through receipts, issues, movements and stock counts, and expose discrepancies immediately.

Data to model

- Warehouses, zones, aisles and bin locations.
- Products with a code, a unit and a storage requirement.
- Stock: the quantity of a product held in a bin.
- Goods receipts from suppliers, with lines and quantities.
- Goods issues against customer orders, with lines and quantities.
- Stock movements between bins.
- Stock counts and the differences they reveal.
- Staff and the tasks assigned to them.

Modeling points to get right

- A bin is identified by warehouse, zone, aisle and position.
- A receipt line is identified only inside its receipt.
- The same product may sit in several bins, and one bin may hold several products.

Queries the database must answer

- Where a given product is currently stored, and in what quantity.
- Products below their reorder level.
- The movements of one product over the last month.
- Bins that were empty during the whole of last week.
- The difference between the counted and the recorded stock in the last stock count.

22. Library Management

Model a library: the titles it holds, the physical copies of each title, who borrows them, and what is overdue.

Objective. Maintain titles, copies, loans, and reservations so the library always knows what is available, what is overdue, and what it is owed.

Data to model

- Titles with an ISBN, a title, a year, and a publisher.
- Authors, and the order in which they appear on a title.
- Copies: several physical copies of one title, each with a copy number and a shelf location.
- Members, their categories (student, staff, external) and their borrowing limits.
- Loans: a copy, a member, a date out, a date due, and a date returned.
- Reservations for a title that is fully on loan.
- Fines for late return, and their payment.

Modeling points to get right

- A copy is identified by its title plus a copy number.
- A member category fixes the loan period and the borrowing limit.
- A reservation is for a title, but a loan is of a copy - explain why.

Queries the database must answer

- The copies of a title currently available.
- The loans of one member that are overdue today.
- The ten most borrowed titles this year.
- Members who have borrowed every title by a given author.
- The total fines outstanding per member category.

23. Hotel Management

Model a hotel group: rooms and their types, reservations, check-in and check-out, the services a guest consumes, and the final bill.

Objective. Handle reservations, room assignment, and billing across a hotel group, with rates that depend on room type and season.

Data to model

- Hotels, floors and rooms; room types with a rate and a capacity.
- Rate plans that vary by season, so the price of a room depends on the date.
- Guests and their identity documents.
- Reservations: a room type, a date range, a number of guests, and a status.
- Stays: the actual room assigned, the check-in and the check-out.
- Services consumed during a stay: restaurant, laundry, minibar.
- Invoices, their lines and the payments received.
- Staff and their assigned duties.

Modeling points to get right

- A room is identified by the hotel plus room number.
- A reservation is made for a room type, and a room is assigned only at check-in.
- The nightly rate depends on the room type and the date, not on the room type alone.

Queries the database must answer

- The rooms free for a date range at one hotel.
- The bill of one stay, with room charges and services.

- Occupancy rate per month per hotel.
- Guests who have stayed at every hotel of the group.
- The service that produces the most revenue per occupied room.

24. Loan Application and Verification

Model the processing of a loan application by a bank: the products offered, the documents a customer must supply, the checks performed, and the decision taken.

Objective. Automate the path from application to decision, showing at any moment which document or verification step an application is waiting for.

Data to model

- Customers with an income, an employment status and existing accounts.
- Loan products with an interest rate, a term range and eligibility rules.
- Applications: customer, product, amount requested, purpose, and date.
- Documents required by a product and the documents actually supplied.
- Verification steps, each performed by an officer with a result and a date.
- Decisions: approved, rejected, or approved with conditions, with a reason.
- Disbursements and the repayment schedule of an approved loan.

Modeling points to get right

- A repayment installment is identified only inside its loan.
- A verification step belongs to an application and a step type.
- One application requires many document types and one document type is required by many products.

Queries the database must answer

- The status of one application with every completed verification step.
- Applications waiting for a document for more than ten days.
- The approval rate per product this year.
- Applications that have supplied every document their product requires.
- Customers with more than one active loan and their total outstanding balance.

25. Parking Booking

Model a parking reservation service: car parks, their bays, the bookings made for a time slot, and the payments and penalties that follow.

Objective. Let drivers reserve a bay for a time slot without conflicts, and give operators a view of occupancy, revenue, and overstay.

Data to model

- Car parks with an address, opening hours and a number of levels.
- Bays: identified inside a car park, with a size and a type (standard, disabled, electric).
- Tariffs by car park, day of week and hour band.
- Users and their registered vehicles.
- Bookings: a bay, a vehicle, a start time and an end time, with a status.
- Entry and exit events recorded by the barrier.
- Payments, cancellations and overstay penalties.

Modelling points to get right

- A bay is identified by car park plus bay number.
- Two bookings for the same bay must not overlap in time — state this constraint and say how you would check it.
- A user may register several vehicles.

Queries the database must answer

- The bays free at a car park between 14:00 and 16:00 tomorrow.
- The bookings of one user in the last month with the amount paid.
- The occupancy rate of each car park by hour of the day.
- Vehicles that exited later than the end of their booking.
- Car parks where every electric bay was booked at least once last week.

Acknowledgement

Several of the application domains in Part 3 were adapted from public collections of database project ideas published by UpGrad and InstantEduHelp. The requirements, the modeling points, the queries, and the assessment scheme in this document are the work of the course instructor and are specific to IT079IU.